Teach Module 45: Infrastructure as Code with Ansible and Terraform

This note is based on Module 45's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

Module 45 Overview

Module 45 covers **Infrastructure as Code with Ansible and Terraform**.

Infrastructure as Code, or IaC, means managing infrastructure using files instead of manual clicking in a cloud console or server UI.

Instead of saying:

```
Create a server.
Install Java.
Open port 8080.
Configure Nginx.
Deploy the application.
```

you describe the environment in code or configuration. Then tools apply that definition consistently.

The two main tools are:

- **Terraform**: creates and manages infrastructure
- **Ansible**: configures machines and applications

Simple mental model:

```
Terraform = build the house
Ansible   = furnish and configure the house
```

Why IaC Matters

Without IaC, environments drift.

One server has Java 17. Another has Java 21. One database has a missing index. One firewall rule was added manually and nobody remembers why.

IaC helps make infrastructure:

- repeatable
- reviewable in Git
- testable
- easier to roll back
- easier to recreate after failure
- consistent across dev, test, staging, and production

For a Java software engineer, this matters because an application depends on more than code. It needs servers, containers, databases, networks, secrets, runtime configuration, logging, and deployment automation.

Terraform Basics

Terraform is mostly **declarative**.

That means you describe the desired final state, and Terraform figures out what must be created, changed, or removed.

Example:

```
resource "aws_instance" "app_server" {
  ami           = "ami-123456"
  instance_type = "t3.micro"

  tags = {
    Name = "java-app-server"
  }
}
```

You are not writing click-by-click instructions. You are saying:

```
I want one server with these properties.
```

Terraform compares your code with the real infrastructure and decides the required actions.

Core Terraform Concepts

Provider

A provider tells Terraform which platform to manage.

Examples:

```
provider "aws" {}
provider "azurerm" {}
provider "google" {}
```

Resource

A resource is something Terraform creates or manages, such as:

- virtual machine
- database
- network
- subnet
- storage bucket
- load balancer
- Kubernetes namespace

Variable

A variable is an input that makes Terraform reusable.

```
variable "environment" {
  type    = string
  default = "dev"
}
```

Output

An output prints a useful value after Terraform runs.

```
output "server_ip" {
  value = aws_instance.app_server.public_ip
}
```

State

Terraform state is Terraform's memory of what it created.

State maps your code to real infrastructure. If state is lost, corrupted, or edited carelessly, Terraform may no longer understand what already exists.

Common commands:

```
terraform init
terraform plan
terraform apply
terraform destroy
```

- `terraform init` prepares the project
- `terraform plan` previews changes
- `terraform apply` makes changes
- `terraform destroy` removes managed infrastructure

Professional habit: always inspect `terraform plan` before applying.

Ansible Basics

Ansible is usually used for **configuration management** and automation.

Terraform might create a Linux VM. Ansible can then install Java, configure users, copy files, install Nginx, start services, and deploy your app.

Example playbook:

```
- name: Configure Java application server
  hosts: app_servers
  become: yes

  tasks:
    - name: Install Java
      apt:
        name: openjdk-17-jdk
        state: present
```

```
  - name: Copy application jar
    copy:
      src: target/app.jar
      dest: /opt/app/app.jar

  - name: Start application service
    systemd:
      name: java-app
      state: started
      enabled: yes
```

Core Ansible Concepts

Inventory

The list of machines Ansible manages.

```
[app_servers]
server1.example.com
server2.example.com
```

Playbook

A YAML file containing automation instructions.

Task

One action inside a playbook.

Module

A reusable unit that performs work, such as installing packages, copying files, managing services, or creating users.

Role

A reusable folder structure for organizing Ansible automation.

Idempotence

Idempotence means you can run the same automation multiple times and get the same final result without breaking things.

Example:

```
state: present
```

means:

```
Install this package if missing.
If it is already installed, do nothing.
```

That is better than blindly running install commands again and again.

Terraform vs Ansible

Use Terraform when you need to provision infrastructure:

- virtual machines
- databases
- networks
- subnets
- cloud storage
- load balancers
- Kubernetes clusters

Use Ansible when you need to configure systems:

- install Java
- configure Nginx
- create users
- copy files
- update config files
- restart services
- deploy application artifacts

Clean mental model:

```
Terraform = infrastructure lifecycle
Ansible   = machine and application configuration
```

How Terraform and Ansible Work Together

A common workflow:

```
1. Terraform creates cloud infrastructure.
2. Terraform outputs server IPs or DNS names.
3. Ansible uses those hosts as inventory.
4. Ansible configures the servers.
5. A CI/CD pipeline runs both in controlled stages.
```

Example:

```
terraform apply
ansible-playbook -i inventory.ini configure-app.yml
```

In a real delivery pipeline:

```
Git commit
Pull request review
Terraform plan
Approval
Terraform apply
Ansible configuration
```

```
Application deployment
Smoke tests
```

Where Java Engineers Fit In

As a Java engineer, you may not own every infrastructure decision, but you should understand enough IaC to answer:

- What runtime does my app need?
- Which environment variables are required?
- Which ports must be open?
- Which database does the app connect to?
- How is the app deployed?
- How does staging differ from production?
- Can I recreate the environment from code?

For Spring Boot apps, infrastructure usually includes:

- Java runtime
- application server or container runtime
- database
- secrets and configuration
- network rules
- logging
- health checks
- deployment pipeline

AI Assistance with IaC

AI tools can help draft Terraform and Ansible, but humans must review the output carefully.

AI may produce code that:

- uses insecure defaults
- exposes ports too broadly
- hardcodes secrets
- uses outdated resource names
- creates expensive infrastructure
- ignores state management
- misses rollback concerns

Good prompt:

```
Create a Terraform configuration for a small dev environment with one Linux VM, no public
database access, and variables for region and instance size.
```

Weak prompt:

```
Make me cloud infra.
```

AI is useful for first drafts. Human review is still required, especially for security, cost, and state.

Practice Exercises

Exercise 1: Terraform Project Structure

Create:

```
iac-practice/
  terraform/
    main.tf
    variables.tf
    outputs.tf
    terraform.tfvars
```

Practice variables and outputs:

```
variable "app_name" {
  type    = string
  default = "java-demo-app"
}

output "application_name" {
  value = var.app_name
}
```

Goal: understand Terraform file organization.

Exercise 2: Terraform Local File Resource

Use Terraform to generate a config file for a Java app:

```
app.name=java-demo-app
server.port=8080
environment=dev
```

Goal: practice Terraform without cloud credentials.

Exercise 3: Terraform Plan vs Apply

Run:

```
terraform init
terraform plan
terraform apply
```

Then change a variable and run `terraform plan` again.

Goal: see how Terraform detects changes.

Exercise 4: Terraform State Inspection

Run:

```
terraform state list
terraform show
```

Goal: understand that Terraform tracks managed resources through state.

Exercise 5: Environment Variable Files

Create:

```
dev.tfvars
test.tfvars
prod.tfvars
```

Then run:

```
terraform plan -var-file="dev.tfvars"
terraform plan -var-file="prod.tfvars"
```

Goal: use one Terraform project for multiple environments.

Exercise 6: Ansible Inventory

Create:

```
[app_servers]
localhost ansible_connection=local
```

Goal: understand how Ansible knows which machines to manage.

Exercise 7: First Ansible Playbook

```
- name: Basic server check
  hosts: app_servers

  tasks:
    - name: Show message
      debug:
        msg: "Running configuration for local app server"
```

Goal: understand playbook structure.

Exercise 8: Generate Java App Config with Ansible

```
- name: Create Java app config
  hosts: app_servers

  tasks:
    - name: Write application config
      copy:
```

```
        dest: ./application.properties
        content: |
          spring.application.name=java-demo-app
          server.port=8080
          app.environment=dev
```

Goal: practice configuration management.

Exercise 9: Practice Idempotence

Run the same Ansible playbook twice.

The first run may report `changed`. The second should report fewer or no changes.

Goal: understand idempotence.

Exercise 10: Terraform + Ansible Workflow

Use Terraform to generate an inventory file:

```
[app_servers]
localhost ansible_connection=local
```

Then use Ansible to read that inventory and configure the app.

Goal: understand how Terraform and Ansible can work together.

Module 45 Lab

Use the existing lab:

```
labs/Week 5 - DevOps, CI-CD and OpenShift/module-45/lab45/LAB-45-GUIDE.md
```

Windows guide:

```
labs/Week 5 - DevOps, CI-CD and OpenShift/module-45/lab45/LAB-45-WINDOWS.md
```

Starter folder:

```
labs/Week 5 - DevOps, CI-CD and OpenShift/module-45/lab45/starter
```

Lab Goal

Practice Infrastructure as Code with Terraform and Ansible by creating a simulated CRM infrastructure setup:

- Terraform defines infrastructure structure, variables, providers, outputs, and plan validation.
- Ansible defines configuration steps for an application environment.
- You review AI-generated IaC instead of trusting it blindly.
- You document security, state, cost, and idempotence decisions.

Copy Starter on Windows

From this folder:

```
labs/Week 5 - DevOps, CI-CD and OpenShift/module-45/lab45
```

run:

```
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\java-bootcamp\examples\lab45-crm" | Out-Null
Copy-Item -Recurse -Force ".\starter\*" "$env:USERPROFILE\java-bootcamp\examples\lab45-crm\"
cd $env:USERPROFILE\java-bootcamp\examples\lab45-crm
```

Main Lab Tasks

1. Complete Terraform files under:

```
infra/terraform/
```

2. Complete Ansible playbook under:

```
infra/ansible/site.yml
```

3. Fill placeholders only in:

```
terraform.tfvars.example
inventory.example.yml
docs/ai-iac-review.md
```

4. Run validation commands when Terraform and Ansible are installed:

```
cd infra\terraform
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
terraform plan -var="environment=dev" -out=tfplan
terraform show -no-color tfplan
```

Then Ansible:

```
cd ..\ansible
ansible-playbook --syntax-check -i ..\..\inventory.example.yml site.yml
```

Deliverables

Submit or check:

- Terraform files
- Ansible playbook
- `terraform.tfvars.example` with no secrets
- `inventory.example.yml`
- `docs/ai-iac-review.md`
- validation evidence or notes explaining unavailable tools

Quick Knowledge Check

1. What does Terraform usually manage?
2. What does Ansible usually manage?
3. Why is Terraform state important?
4. What does idempotence mean?
5. In a Java app deployment, which tool would you use to create a database: Terraform or Ansible?

Short Summary

Module 45 is about making infrastructure predictable.

Terraform creates and manages the infrastructure. Ansible configures machines and applications. Both should live in Git, be reviewed like application code, and be treated carefully because mistakes can affect security, cost, and production stability.